

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 15



FSA State Minimization

$$L = (ab \cup ba)^*$$

And the FSA accepting L is M:

Consider the states q_7 & q_8

Observe:

These states are *unreachable*

Because there is no path from the start state to it.

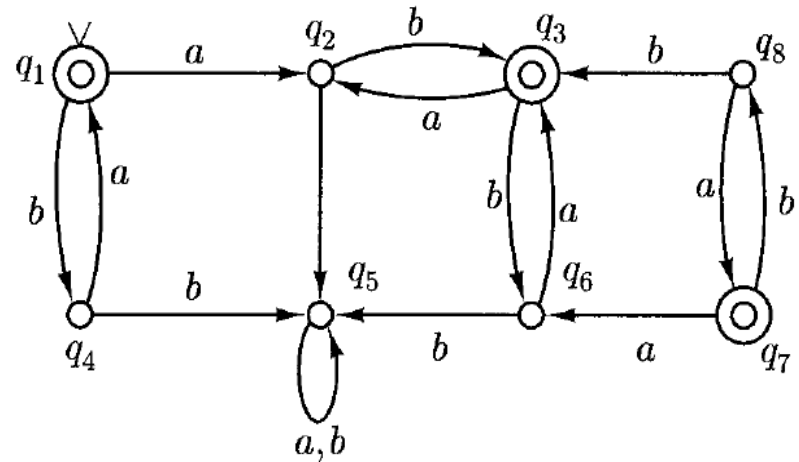
Simplest State Optimization for an DFA:

Remove all *unreachable states* & all transitions in and out of them.

Next Observe

For the states q_4 & q_6 precisely the *same strings* lead M to *acceptance*.

We may think these as '*equivalent*' and **merge** them.



FSA State Minimization

Building a *Minimal DFA* for a *Regular* Language

Let $L \subseteq \Sigma^*$ be a language & let $x, y \in \Sigma^*$.

We say that x & y are *indistinguishable/equivalent* wrt L , denoted as $x \approx_L y$,

if for all $z \in \Sigma^*$, the following is true : $xz \in L \Leftrightarrow yz \in L$

Intuitively:

$x \approx_L y$ if either both strings belong to L or neither is in L

& Moreover, appending any fixed string to both x & y results in two strings that are either both in L or both not in L .

Note: 1. $x \approx_L y$ is an *equivalence relation* (Exercise)

2. If x and y are *distinguishable*, then there exists *at least one string* z such that one but not both of xz and yz is in L .

FSA State Minimization

Let $L = (ab \cup ba)^*$. To determine the equivalence classes of $\approx_1$

The FSA accepting L is :

We *begin* by creating a first class [1] and arbitrarily assigning empty string e to it.

Easy to compute (by observation)

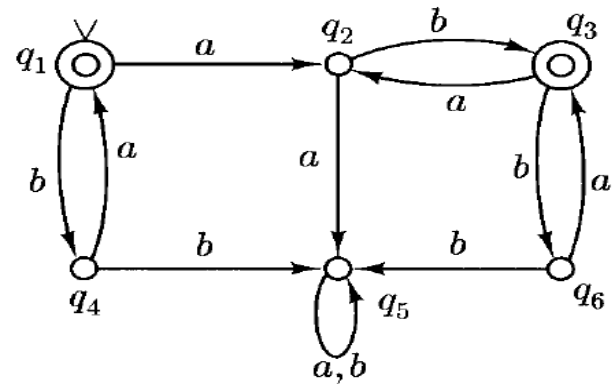
the classes by considering the FSA

$[1] = [e] = L$

We observe a is *distinguishable* from e , so we create a new class [2]
 $= [a] = La$

Next, We observe b is *distinguishable* from both e & a , so we create a new class [3] = [b] = Lb

Finally $[aa] = L(aa \cup bb) \Sigma^*$



Note $[e] \cup [a] \cup [b] \cup [aa] = \Sigma^*$
& they are all *disjoint*

FSA State Minimization

Next we define a *relation* on Σ^* using a given DFA M .

Definition:

Let $M = (K, \Sigma, \delta, s, F)$ be DFA & let $x, y \in \Sigma^*$

We say that x & y are *indistinguishable/equivalent* wrt M , denoted as $x \sim_M y$ if there is a state q such that

$(s, x) \vdash_M^* (q, e)$ and $(s, y) \vdash_M^* (q, e)$.

Intuitively, both x & y *drive* M from s to the same state.

Note:

1. $\sim_M$ is an *equivalence relation* (Exercise)
2. Equivalence classes can be *identified* by the states of M .

For $q \in K$, E_q is the set of (the equivalence class corresponding to q) all strings in Σ^* that drives M from s to the state q .

FSA State Minimization

Consider the DFA M

Note

$\sim_M$ has six different classes

$$E_{q_1} = (ba)^*$$

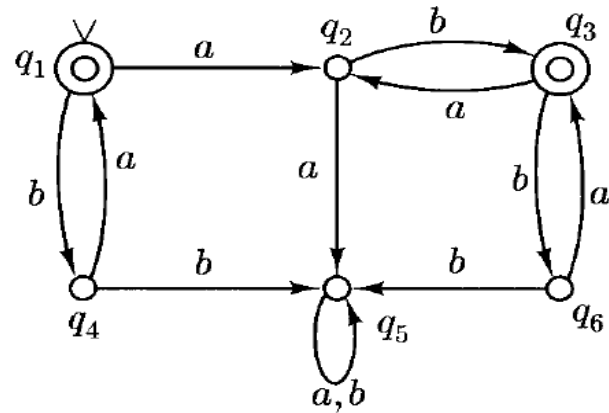
$$E_{q_2} = La$$

$$E_{q_3} = (ba)^*abL$$

$$E_{q_4} = b(ba)^*$$

$$E_{q_5} = L(aa \cup bb) \Sigma^*$$

$$E_{q_6} = (ba)^*abLb$$



Note

- Classes form a **partition** of Σ^*
- Both $\approx_L$ and $\sim_M$ are **partitions** of Σ^*

E_{q_1} & E_{q_3} are contained in $[e]$

$E_{q_2} \subseteq [a]$

E_{q_4} & E_{q_6} are contained in $[b]$

$E_{q_5} \subseteq [aa]$

We say the **partition** due to $\sim_M$ is an

refinement of the **partition** due to $\approx_L$

FSA State Minimization

Result: For any DFA $M = (K, \Sigma, \delta, s, F)$ and any strings $x, y \in \Sigma^*$,
if $x \sim_M y$, then $x \approx_{L(M)} y$

We say that $\sim_M$ is a *refinement* of $\approx_{L(M)}$

Note

L be a regular language & say for DFA M , $L(M) = L$

By the result above, the *number of states* in M must be *at least as large as* the *number of equivalence classes* of $L(M)$ under $\approx$

In other words, the *number of equivalence classes* of $L(M)$ under $\approx$ is a *lower bound* on the *number of states* of any DFA accepting $L(M)$.

We show that this *lower bound* is *achieved*.

Myhill-Nerode Theorem

Myhill-Nerode Theorem

Let $L \subseteq \Sigma^*$ be a *regular* language. Then there exists DFA with *precisely as many states* as there are *equivalence classes* in $\approx_L$ that accepts L .

Proof: We define DFA $M = (K, \Sigma, \delta, s, F)$ as follows:

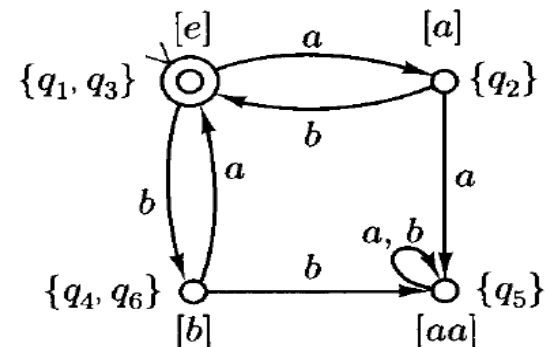
$K = \{[x] : x \in \Sigma^*\}$, the set of equivalence classes in $\approx_L$

$s = [e]$, the equivalence classes of *empty string* e in $\approx_L$

$F = \{[x] : x \in L\}$

Minimal DFA corresponding to $L = (ab \cup ba)^*$

For any $[x] \in K$ and any $a \in \Sigma$,
define $\delta([x], a) = [xa]$



Characterization of Regular Languages

Corollary:

A language L is regular if and only if $\approx_L$ has finitely many equivalence classes.